



HASHDIT

HashDit

Smart Contract Security

ListaDao ClisBNBLaunchPoolDistributor Update Security Audit Report

ListaDao

12 Aug 2026 – 13 Aug 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	12 Aug 2026 –13 Aug 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-token/pull/123
Commit	feat/launchpool-topup-epoch

Smart Contract List

No.	Contract Name	Verdict	Details
1	ClisBNBLaunchPoolDistributor.sol	Medium	[M01]
			[M02]
			[L01]
			[I01]
			[I02]
			[I03]
			[I04]
			[I05]
			[I06]

Findings

[M01] topUpEpoch solvency check is aggregate rather than per epoch

Contract ClisBNBLaunchPoolDistributor.sol

Severity Medium

Description

The ClisBNBLaunchPoolDistributor.topUpEpoch() function validates solvency with require(_balanceOf(token) >= totalUnclaimedAmount[token]). This checks protocol wide solvency for the token, which is the correct invariant for a fungible balance shared across epochs, but it does not prove that the specific epoch being topped up is individually funded.

JavaScript

```
require(_balanceOf(epoch.token) >= totalUnclaimedAmount[epoch.token],  
"Insufficient funds");
```

Description

Impact

When there are >= 2 epochs backed by the same token, a surplus held by a sibling epoch can satisfy the check even though the admin never funded this epoch's delta.

Conversely, a deficit in a sibling epoch can cause the check to fail and block a legitimate top up of a different epoch. This is not exploitable because the function is admin gated and allows no theft, but a successful topUpEpoch call must not be interpreted as proof that the individual epoch is fully funded.

Note that the behavior is untested since all existing tests use a single epoch.

Consider the following:

Recommendation

- Track and validate funding on a per epoch basis in addition to the aggregate token level check, so that a top up confirms the specific epoch has received its required delta rather than relying on the combined balance across all epochs sharing the token.
- If not, operationally, at any given moment of time, there should only be one epoch present.

Status

Acknowledged, ListaDao has indicated that operationally, they will ensure that only one epoch per token is live at a time, which makes the two checks equivalent in practice.

For multi-epoch scenarios with identical epoch tokens, ListaDao accepts it as design decision that a successful topUpEpoch is not proof the epoch is individually funded.

[M02] topUpEpoch can be called on a not yet started epoch, enabling surplus creation and cross epoch solvency coupling

Contract ClisBNBLaunchPoolDistributor.sol

Severity Medium

Description

The ClisBNBLaunchPoolDistributor.topUpEpoch() function only bounds the upper edge of an epoch's window, requiring block.timestamp <= endTime, and never checks that the epoch has started. Because a pre start epoch always has endTime > startTime > now, the end time check always passes for it, so any epoch whose startTime > now can be topped up.

This is functionally reversible on its own for a singular epoch, since no claims can occur before an epoch starts and a later revokeEpoch subtracts the full unclaimedAmount, including the added amount, so the accounting nets back to zero.

JavaScript

```
function topUpEpoch(uint64 _epochId, uint256 _newTotalAmount)
external onlyRole(DEFAULT_ADMIN_ROLE) {
    Epoch storage epoch = epochs[_epochId];
    require(epoch.merkleRoot != bytes32(0), "Invalid epochId");
    require(block.timestamp <= epoch.endTime, "Epoch ended");
    require(_newTotalAmount > epoch.totalAmount, "Not an increase");

    uint256 oldTotalAmount = epoch.totalAmount;
    uint256 added = _newTotalAmount - oldTotalAmount;

    epoch.totalAmount = _newTotalAmount;
    epoch.unclaimedAmount += added;
    totalUnclaimedAmount[epoch.token] += added;

    require(_balanceOf(epoch.token) >=
totalUnclaimedAmount[epoch.token], "Insufficient funds");

    emit TopUpEpoch(_epochId, epoch.token, oldTotalAmount,
_newTotalAmount, added);
}
```

Description

Impact

The problem is the side effects this allows in a multi epoch token.

1. Topping up a pre-start epoch immediately increases the shared totalUnclaimedAmount[token] and enforces the aggregate solvency requirement _balanceOf(token) >= totalUnclaimedAmount[token], pulling that

epoch's funding forward in time. In a token with multiple epochs this raises the pooled obligation right away, which can block a legitimate top up of an already active sibling epoch or force the admin to provision more funds than expected, coupling an epoch that has not started to the solvency of ones that have.

2. `revokeEpoch` only adjusts accounting and deletes the epoch, it does not return any tokens. Topping up a pre start epoch and then revoking it leaves the added tokens in the contract as balance above `totalUnclaimedAmount`, recoverable only through `adminTransfer`. That unearmarked surplus is exactly what lets a later top up pass its aggregate solvency check without the admin actually funding that epoch's delta, so this issue manufactures the masking surplus that undermines the per epoch funding guarantee.
3. Finally, the clean reversal is only guaranteed if the roles cooperate, since `topUpEpoch` is restricted to `DEFAULT_ADMIN_ROLE` while `revokeEpoch` is restricted to `OPERATOR`. There is no theft or direct claimer loss, but the behavior is semantically incorrect, since a pre start epoch would normally be revoked and recreated with the correct amount, and it compounds the aggregate solvency weakness in multi epoch tokens.

Recommendation

Consider restricting `topUpEpoch` to epochs that are within their active claim window by adding a lower bound such as `require(block.timestamp >= epoch.startTime, "Epoch not started")` alongside the existing `block.timestamp <= endTime` check, so that pre start epochs must be corrected through `revoke` and `recreate` rather than through a top up.

Operationally, consider only having a singular epoch at any given moment of time.

Status

Fixed, now restricts epoch topups to only after the epoch has started. Additionally, as mentioned in [M01], ListaDao has indicated that operationally, they will ensure that only one epoch per token is live at a time.

[L01] `setEpochMerkleRoot` does not verify that `totalAmount` equals the sum of the merkle leaf amounts

Contract ClisBNBLaunchPoolDistributor.sol

Severity Low

Description

Description

The `ClisBNBLaunchPoolDistributor.setEpochMerkleRoot()` function accepts a `totalAmount` that is never checked against the sum of the reward amounts encoded in the merkle tree. The contract has no way to recompute that sum on chain, since only the root is stored, so an epoch can be published where `totalAmount` is below the true sum of all leaves.

When this happens, `claim()` works for early users but decrements `epoch.unclaimedAmount` past zero once cumulative claims exceed the low `totalAmount`. The subtraction reverts, and every remaining user is blocked from

claiming. This is the exact condition that occurred on the pendle-v3 launchpool epochs and the reason `topUpEpoch()` was introduced.

JavaScript

```
function setEpochMerkleRoot(uint64 _epochId, bytes32 _merkleRoot,
address _token, uint256 _startTime, uint256 _endTime, uint256
_totalAmount)
    external onlyRole(OPERATOR)
{
    require(_epochId == nextEpochId, "Invalid epochId");
    require(_merkleRoot != bytes32(0), "Invalid merkle root");
    require(_startTime > block.timestamp, "Invalid start time");
    require(_endTime > _startTime, "Invalid end time");
    require(_totalAmount > 0, "Invalid total amount");

    uint64 currentEpochId = nextEpochId++;
    Epoch storage epoch = epochs[currentEpochId];
    epoch.merkleRoot = _merkleRoot;
    epoch.token = _token;
    epoch.startTime = _startTime;
    epoch.endTime = _endTime;
    epoch.totalAmount = _totalAmount;
    epoch.unclaimedAmount = _totalAmount;
    totalUnclaimedAmount[_token] += _totalAmount;

    emit UpdateEpoch(currentEpochId, epoch.merkleRoot, epoch.token,
epoch.startTime, epoch.endTime, epoch.totalAmount);
}
```

For a plain launchpool (i.e. hold clisBNB in your wallet), snapshotting clisBNB balance is simple, however, pendle likely breaks one of those assumptions making it difficult to set an accurate merkle root and associated amounts:

1. The clisBNB could have been fragmented across layered positions. Once clisBNB enters Pendle it becomes SY, then splits into PT + YT, and PT/SY sit in an AMM LP (and possibly a gauge, or are re-lent in a money market). A user's clisBNB exposure is spread across several of these simultaneously, so attribution requires indexing all venues and correctly mapping each wrapper back to underlying clisBNB.
2. The clisBNB-equivalent of PT/YT is time- and rate-dependent. PT accretes toward par (1 clisBNB) at maturity while YT decays, so their "clisBNB value" is a function of the implied yield and time-to-maturity, not a fixed 1:1. Pricing them requires a consistent valuation curve, and a small modeling choice difference between the numerator and denominator paths shifts the sum.
3. Positions move every block. A time-weighted average balance over an epoch has to integrate a continuously changing AMM composition (trades, LP mints/burns). TWAB over a fast-moving pool is where rounding and sampling errors accumulate.

Impact

The important point is that `topUpEpoch()` only resolves the problem after it has already reached production. It does not prevent an underfunded epoch from being created in the first place, and it must be executed by a privileged role before the epoch `endTime`, so the same misconfiguration can recur and each occurrence needs a manual, time bound repair. The root cause, the missing reconciliation between the off chain per user amounts and the on chain `totalAmount`, remains open. The comment on the one week dispute period does not help here, because the underfunding is not visible until enough users have claimed, which typically happens late in the claim window.

Consider making the invariant `totalAmount == sum of leaf amounts` verifiable on chain rather than trusted. If not, off-chain logic needs to be more comprehensive for more complex launch pool rewards distribution (e.g.. including a buffer).

Recommendation

Commit the leaf sum into the merkle tree, for example as a dedicated total leaf or a signed total field, and have `setEpochMerkleRoot()` require that the supplied `totalAmount` matches that committed value before activating the epoch. As an additional layer, fund the epoch by pulling exactly `totalAmount` at creation so an epoch can never be activated below its configured total

Alternatively, consider a design improvement migration for `ClisBNBLaunchPoolDistributor` in the future highlighted in [I05].

Status

Acknowledged, no fix implemented. ListaDao has indicated that off-chain, the `totalAmount` summed from the same tree that was just built, reconciled on-chain before `startTime`. Additionally, a future design improvement highlighted in [I05] will be considered.

[I01] `adminTransfer` lacks a solvency floor and does not guarantee funds reserved for claimers

Contract `ClisBNBLaunchPoolDistributor.sol`

Severity **Informational**

Description

The `ClisBNBLaunchPoolDistributor.adminTransfer` function moves any amount out of the contract without any restriction. This lets the admin withdraw an amount that brings the token balance below `totalUnclaimedAmount[token]`, which would leave insufficient funds to cover pending claims and brick all outstanding claims for that token mid epoch.

Description

JavaScript

```
/**
 * @dev Transfer the given amount to the admin
 * @param _amount Amount to transfer
 */
```

```
function adminTransfer(address _token, uint256 _amount) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_amount > 0, "Invalid amount");
    _transferTo(msg.sender, _token, _amount);
}
```

Impact

Because there is no on chain check, every solvency guarantee in the contract, including the one that topUpEpoch establishes, holds only as long as the admin chooses to respect it. This is a by design trust assumption on the admin, but the absence of an enforced reserve means claimers have no on chain protection.

Note that this issue is pre-existing and was not introduced by PR #123.

Consider adding an on chain solvency floor to adminTransfer that enforces $\text{balance} - \text{amount} \geq \text{totalUnclaimedAmount}[\text{token}]$, reverting any withdrawal that would drop the balance below the total amount still owed to claimers.

Recommendation

Withdrawals of surplus funds should instead go through collectUnclaimed, which is the intended path for removing amounts that are not reserved for claimers. If enforcing this on chain is not desired, explicitly document the admin trust assumption so that the lack of a guaranteed reserve for claimers is clear.

Status

Acknowledged, the 3/6 multisig holding the DEFAULT_ADMIN_ROLE is trusted in this case to only utilize adminTransfer as a last resort to withdraw launchpool funds.

[I02] claim decrements the global unclaimed total before the per epoch amount

Contract ClisBNBLaunchPoolDistributor.sol

Severity **Informational**

Description

Description

In ClisBNBLaunchPoolDistributor.claim(), the reward accounting is reduced in the order of the contract wide counter first and the per epoch counter second, meaning $\text{totalUnclaimedAmount}[\text{token}] -= \text{_amount}$ runs before $\text{epoch.unclaimedAmount} -= \text{_amount}$.

Because $\text{totalUnclaimedAmount}[\text{token}]$ is the sum of the unclaimed amounts across every epoch sharing that token, it is always $\geq$ the individual $\text{epoch.unclaimedAmount}$. In an under funded epoch, where a tail user's claim amount exceeds the remaining per epoch balance, the per epoch value is the binding constraint.

JavaScript

```
totalUnclaimedAmount[epoch.token] -= _amount;  
epoch.unclaimedAmount -= _amount;
```

Impact

When there are ≥ 2 epochs on the same token, a sibling epoch's surplus lets the global decrement pass while the per epoch decrement is the line that actually catches the shortfall and reverts. When there is a single epoch on the token, both counters are equal and the global line reverts first, so the ordering makes no observable difference. In all cases the arithmetic is checked under Solidity 0.8, so whichever line underflows reverts the entire transaction atomically and rolls back the earlier decrement. There is no state corruption and no exploit, and the situation is resolved for legitimate claims once `topUpEpoch` raises the epoch caps. The only effect of the ordering is which line produces the revert.

Recommendation

Optionally consider reordering the two decrements so the binding per epoch constraint is applied first, subtracting `epoch.unclaimedAmount -= _amount` before `totalUnclaimedAmount[token] -= _amount`.

Status

Acknowledged, deferred to next version as a potential improvement, no direct impact on core contract functionality.

[I03] adminTransfer does not emit an event

Contract ClisBNBLaunchPoolDistributor.sol

Severity **Informational**

Description

The `ClisBNBLaunchPoolDistributor.adminTransfer()` function moves tokens or BNB out of the contract to the admin but does not emit any event when it executes. Every other state changing action in the contract, such as updating an epoch, revoking an epoch, collecting unclaimed rewards, topping up an epoch, and claiming, emits a dedicated event, so this function is inconsistent with the rest of the contract.

Description

JavaScript

```
/**  
 * @dev Transfer the given amount to the admin  
 * @param _amount Amount to transfer  
 */  
function adminTransfer(address _token, uint256 _amount) external  
onlyRole(DEFAULT_ADMIN_ROLE) {  
    require(_amount > 0, "Invalid amount");  
    _transferTo(msg.sender, _token, _amount);  
}
```

	Impact Without an emitted event, admin withdrawals leave no structured on chain record, which makes off chain monitoring, accounting, and after the fact auditing of privileged fund movements harder. Since this function can reduce the contract balance below what is owed to claimers, the absence of an event weakens transparency around a sensitive privileged operation.
Recommendation	Consider adding an event that records the admin withdrawal, including the token address, the amount transferred, and the recipient, and emit it inside adminTransfer after the transfer.
Status	Acknowledged, deferred to next version as a potential improvement, no direct impact on core contract functionality.

[I04] Missing indexed parameters in ClisBNBLaunchPoolDistributor events

Contract	ClisBNBLaunchPoolDistributor.sol
Severity	Informational
Description	Description None of the events declared in ClisBNBLaunchPoolDistributor, including UpdateEpoch, RevokeEpoch, CollectUnclaimed, Claimed, and TopUpEpoch, mark any of their parameters as indexed. Because event parameters are only stored in the log topics when they are indexed, off chain services, indexers, and monitoring tools cannot efficiently filter these events by key fields such as epochId, token, or account. <pre> JavaScript event UpdateEpoch(uint64 epochId, bytes32 merkleRoot, address token, uint256 startTime, uint256 endTime, uint256 amount); event RevokeEpoch(uint64 epochId, address token, uint256 totalAmount, uint256 unclaimedAmount); event CollectUnclaimed(uint64 epochId, address token, uint256 totalAmount, uint256 unclaimedAmount); event Claimed(address account, uint64 epochId, address token, uint256 amount); event TopUpEpoch(uint64 epochId, address token, uint256 oldTotalAmount, uint256 newTotalAmount, uint256 addedAmount); </pre> Impact Instead they must fetch and decode the full data payload of every emitted log to locate relevant entries. This makes it harder to track a specific epoch, token, or user, degrades

	the experience of dashboards and subgraphs that rely on topic based filtering, and increases the cost of building reliable off chain accounting around the distributor.
Recommendation	Consider adding the indexed keyword to the parameters that are most useful for filtering across these events.
Status	Acknowledged, deferred to next version as a potential improvement, no direct impact on core contract functionality.

[I05] ClisBNBLaunchPoolDistributor design improvements

Contract	ClisBNBLaunchPoolDistributor.sol
Severity	Informational
Description	Description The ClisBNBLaunchPoolDistributor contract distributes rewards through a Merkle based epoch model but omits a broad set of protections than other comparable Distributor (e.g. LisAsterDistributor, LendingRewardsDistributor) contract implements. Individually, these range from architectural to minor, but together they leave the contract dependent on operator and admin discipline where an equivalent contract enforces the same guarantees on chain. The gaps are as follows. 1. There is no inflow to allocation solvency invariant. The contract tracks only totalUnclaimedAmount[token] and never records how much reward was actually funded, so allocation is a trust assumption rather than an enforced relationship. For example, LisAsterDistributor enforces totalClaimed <= totalAllocated <= totalNotified, recording every inflow and preventing a root from allocating more than has been notified. This missing invariant could have prevented the under set cap problem, and of the topUpEpoch machinery that exists to patch it along with that function's own aggregate solvency check, pre start top up, and increase only rigidity issues. 2. Merkle root updates are single step. setEpochMerkleRoot takes effect immediately when called by the operator, with no on chain dispute window, while the referenced one week dispute period is not enforced in code and only pre start revokeEpoch exists. For example, LisAsterDistributor uses a two step, time locked flow of staging a pending root, waiting a minimum period, and then accepting it, with a manager veto to revoke a wrong or malicious root during the wait. A comparable time lock and veto would give a guaranteed detection window for ClisBNBLaunchPoolDistributor to detect incorrect merkle root settings. The side effect is that role separation is coarse. The contract exposes only DEFAULT_ADMIN_ROLE and OPERATOR, whereas LisAsterDistributor separates duties across admin, manager, bot, and pauser roles, following least privilege more closely.

3. The contract has no pause mechanism. For example, LisAsterDistributor is pausable with a pauser role that can trip the contract in an incident and a manager role required to resume, allowing claim to be halted during an emergency.
4. claim has no reentrancy guard. The existing checks effects interactions ordering, setting claimed and decrementing counters before the transfer, prevents same epoch double claiming, but the low level native BNB call to an arbitrary recipient warrants a guard as defense in depth, which LisAsterDistributor applies to all user facing mutating functions.

Recommendation Consider a migration to a more hardened and an improved designed ClisBNBLaunchPoolDistributor for future distribution of launchpool rewards.

Status Acknowledged, deferred to next version as a potential improvement, no direct impact on current core contract functionality.

[I06] Under set epoch cap cannot be repaired for shortfalls discovered after the claim period has passed

Contract ClisBNBLaunchPoolDistributor.sol

Severity **Informational**

Description

The ClisBNBLaunchPoolDistributor contract repairs an epoch whose totalAmount was set below the true sum of its merkle leaves through topUpEpoch, but that repair is only possible while block.timestamp <= endTime.

In the scenario where the problem is discovered after the claim period e.g Once endTime has passed for such an epoch, there is no on chain path left and a new epoch would have to be created. This can happen when large volume of claims are only made towards the end of the epoch.

Description

JavaScript

```
event UpdateEpoch(uint64 epochId, bytes32 merkleRoot, address token,
uint256 startTime, uint256 endTime, uint256 amount);
```

```
event RevokeEpoch(uint64 epochId, address token, uint256
totalAmount, uint256 unclaimedAmount);
```

```
event CollectUnclaimed(uint64 epochId, address token, uint256
totalAmount, uint256 unclaimedAmount);
```

```
event Claimed(address account, uint64 epochId, address token,
uint256 amount);
```

```
event TopUpEpoch(uint64 epochId, address token, uint256
oldTotalAmount, uint256 newTotalAmount, uint256 addedAmount);
```

Impact

The claim function reverts because the epoch is inactive, topUpEpoch reverts because the epoch has ended, collectUnclaimed reverts because unclaimedAmount is already zero after earlier claimers drained the under set cap, and revokeEpoch reverts because the epoch has started. Users who tried to claim in time but were rejected by the contract are permanently unable to claim, and because the shortfall was never funded there is nothing for the admin to redistribute through the existing functions. The impact is limited to distributions that were misconfigured at creation and depends on operator error rather than an attacker, which is why this is informational, but the consequence when it occurs is a permanent loss of access for affected claimers.

No code changes recommended. This finding serves as an additional note to perform sufficient off-chain merkle root settings, or consider a design improvement towards the ClisBNBLaunchPoolDistributor in general.

Recommendation

In the future, if this happens, a new epoch would be required to be set and/or we need to consider an admin only function extending the epoch end time, e.g. gating collectUnclaimed on endTime + REMEDIATION_PERIOD instead of endTime, and allow topUpEpoch plus a bounded endTime extension during that remediation period. This allows funds to not be swept immediately, so there is always a fixed buffer (say 7 days) in which the team can extend the claim window, top up the cap, and let blocked users claim.

Status

Acknowledged, deferred to next version as a potential improvement, no direct impact on current core contract functionality.